

STAT 4710/5710: Homework 2

David Dai

Sep 25, 2025

Contents

Instructions	2
1 Case study: spline regression model selection (64 points)	4
1.1 Import data (2 points)	4
1.2 Tidy (2 points)	4
1.3 Explore (8 points)	5
1.4 Prepare for modeling (1 point)	7
1.5 Validation set approach to model selection (33 points)	7
1.5.1 Data splitting (3 points)	7
1.5.2 Estimate MSE given a df (15 points)	8
1.5.3 Model selection based on estimated MSE (10 points)	8
1.5.4 Fit chosen model and assessment (5 points)	9
1.6 Cross validation model selection (18 points)	10
1.6.1 Model selection (11 points)	10
1.6.2 Final fit (5 points)	12
1.6.3 Compare with validation set approach (2 points)	13
2 Case study: kNN classification with Default data (36 points)	14
2.1 Data split (2 points)	14
2.2 Class imbalance and baseline (6 points)	14
2.3 kNN classification (28 points)	15
2.3.1 Load results	15
2.3.2 Compute error metrics in classification (4 points)	15
2.3.3 Plot metrics vs. K (6 points)	15
2.3.4 Model selection and improvement (conceptual) (8 points)	17
2.3.5 Model complexity (conceptual) (10 points)	17

Instructions

Materials and collaboration

The policy on allowed materials and collaboration is as stated on the Syllabus:

“Students are permitted to work together on homework assignments, but must write up and submit solutions individually. In particular, students may not copy each others’ solutions. Students may consult all course materials, textbooks, the internet, or AI tools (e.g. ChatGPT) to complete their homework. Students may not use solutions to problems that may be available online and/or from past iterations of the course. For each homework, students must disclose all classmates with whom they collaborated, which AI tools they used, and how they used them. Failure to do so will result in a 10-point penalty.”

In accordance with this policy,

Please disclose all classmates with whom you collaborated: NA

Please disclose which AI tools you used, and how you used them: I used ChatGPT for R code grammar example

Failure to answer the above questions will result in a 10-point penalty.

Writeup

Use `homework-2-template.Rmd` as a starting point for your writeup, adding your solutions after **“Solution”**. Add your R code using code chunks and add your text answers using **“bold text”**. Consult the [preparing reports guide](#) for guidance on compilation, creation of figures and tables, and presentation quality. If the instructions ask you to “print a tibble”, you should print the tibble directly.

Programming

We encourage you to follow the `tidyverse` paradigm for data visualization, manipulation, and wrangling, as in the first homework, to practice tidyverse workflows (e.g., `filter`, `mutate`, `group_by`, `summarize`, `join`, `ggplot2`).

Students who strongly prefer to use Python for homework may do so. If you choose this option, you are **required** to present both code chunks and the corresponding outputs/interpretations in your PDF. In addition, it’s recommended that you:

- Use `pandas` for data manipulation (preferred, since it parallels `dplyr`).
- Use `plotnine` for visualization (preferred, since it parallels `ggplot2`). If using `seaborn`/`matplotlib`, you must still include the required plot elements (labels, 45° line, smoothing curve, facets, etc.).

```
library(tidyverse)
library(splines)
```

Code completion. In some problems, the task is to complete an incomplete code chunk provided. In those problems, in the template, there is `eval = FALSE` in the chunk header to prevent evaluation errors. You should remove `eval = FALSE` after you have completed the code to enable evaluation and printing of results.

Grading

The point value for each problem sub-part is indicated. There are 100 points possible on this homework, evaluated based on the correctness of the numerical results and plots. When a problem asks for a plot:

- **Axes and labels:** Always include informative axis labels and, if appropriate, titles.
- **Required elements:** If the instructions ask for a 45° line, smoothing curve, or facets, these must be included.
- **Clarity:** Points may be deducted if plots are cluttered, unreadable, or missing labels.

Submission

Compile your writeup to PDF and submit to Gradescope.

1 Case study: spline regression model selection (64 points)

1.1 Import data (2 points)

Solution.

```
library(tidyverse)
wage_df = read_csv("wages.csv")

## Rows: 3000 Columns: 10
## -- Column specification -----
## Delimiter: ","
## chr (7): maritl, race, education, region, jobclass, health, health_ins
## dbl (3): year, age, wage
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.

# log transform wage column
wage_df$wage = log(wage_df$wage)
print(wage_df)

## # A tibble: 3,000 x 10
##   year   age maritl   race education region jobclass health health_ins wage
##   <dbl> <dbl> <chr>   <chr> <chr>   <chr> <chr>   <chr>   <chr>   <dbl>
## 1  2006    18 1. Never~ 1. W~ < HS Grad 2. Mi~ 1. Indu~ 1. <=~ 2. No    1.46
## 2  2004    24 1. Never~ 1. W~ College ~ 2. Mi~ 2. Info~ 2. >=~ 2. No    1.43
## 3  2003    45 2. Marri~ 1. W~ Some Col~ 2. Mi~ 1. Indu~ 1. <=~ 1. Yes   1.58
## 4  2003    43 2. Marri~ 3. A~ College ~ 2. Mi~ 2. Info~ 2. >=~ 1. Yes   1.59
## 5  2005    50 4. Divor~ 1. W~ HS Grad   2. Mi~ 2. Info~ 1. <=~ 1. Yes   1.44
## 6  2008    54 2. Marri~ 1. W~ College ~ 2. Mi~ 2. Info~ 2. >=~ 1. Yes   1.56
## 7  2009    44 2. Marri~ 4. O~ Some Col~ 2. Mi~ 1. Indu~ 2. >=~ 1. Yes   1.64
## 8  2008    30 1. Never~ 3. A~ Some Col~ 2. Mi~ 2. Info~ 1. <=~ 1. Yes   1.55
## 9  2006    41 1. Never~ 2. B~ Some Col~ 2. Mi~ 2. Info~ 2. >=~ 1. Yes   1.56
## 10 2004    52 2. Marri~ 1. W~ HS Grad   2. Mi~ 2. Info~ 2. >=~ 1. Yes   1.60
## # i 2,990 more rows
```

1.2 Tidy (2 points)

Solution.

```
# clean the wage_df
wage_df = wage_df |>
  select(wage, age, education) |>
  drop_na() |>
  filter(education %in% c("HS Grad", "College Grad"))

print(wage_df)

## # A tibble: 1,656 x 3
##   wage   age education
```

```
##      <dbl> <dbl> <chr>
## 1  1.43    24 College Grad
## 2  1.59    43 College Grad
## 3  1.44    50 HS Grad
## 4  1.56    54 College Grad
## 5  1.60    52 HS Grad
## 6  1.48    34 HS Grad
## 7  1.54    35 HS Grad
## 8  1.62    39 College Grad
## 9  1.59    54 HS Grad
## 10 1.52    37 College Grad
## # i 1,646 more rows
```

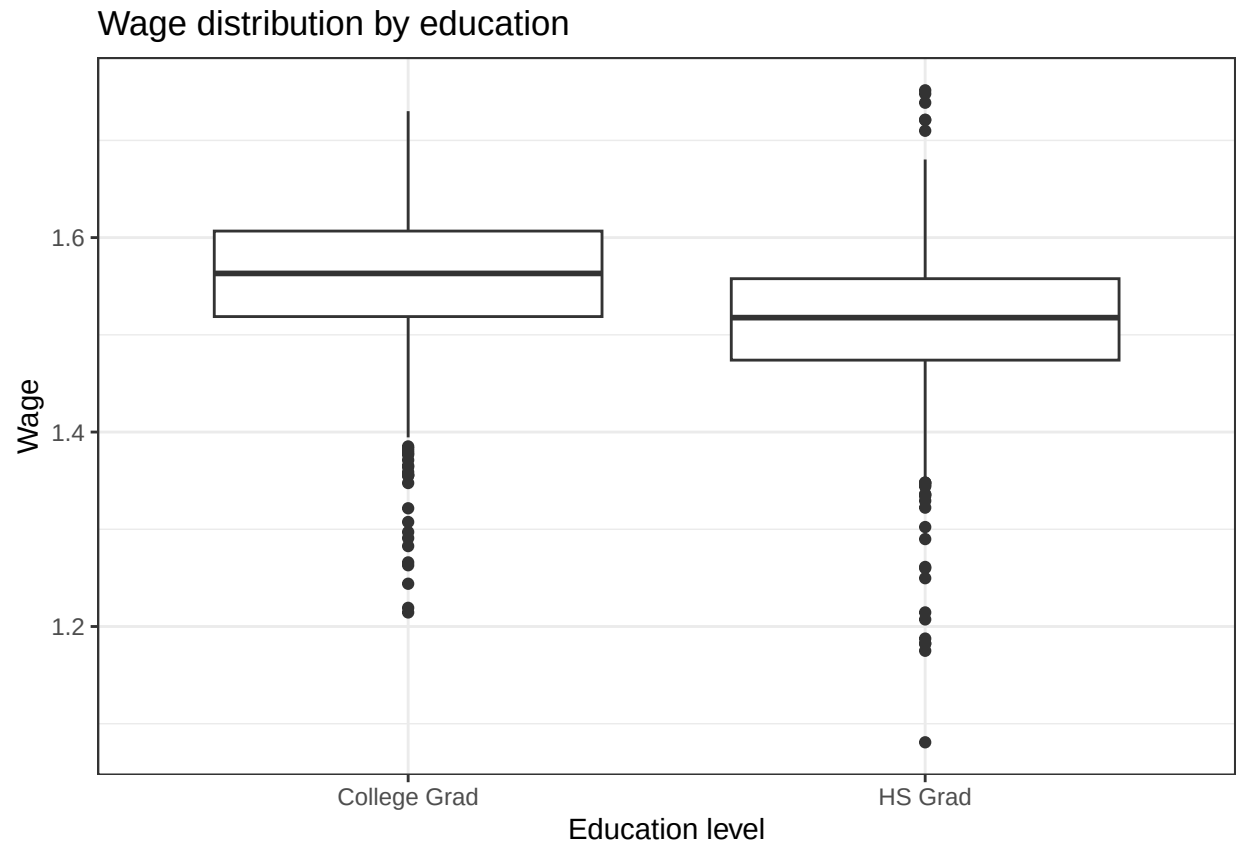
1.3 Explore (8 points)

Solution.

```
# check count of each education level
print(table(wage_df$education))
```

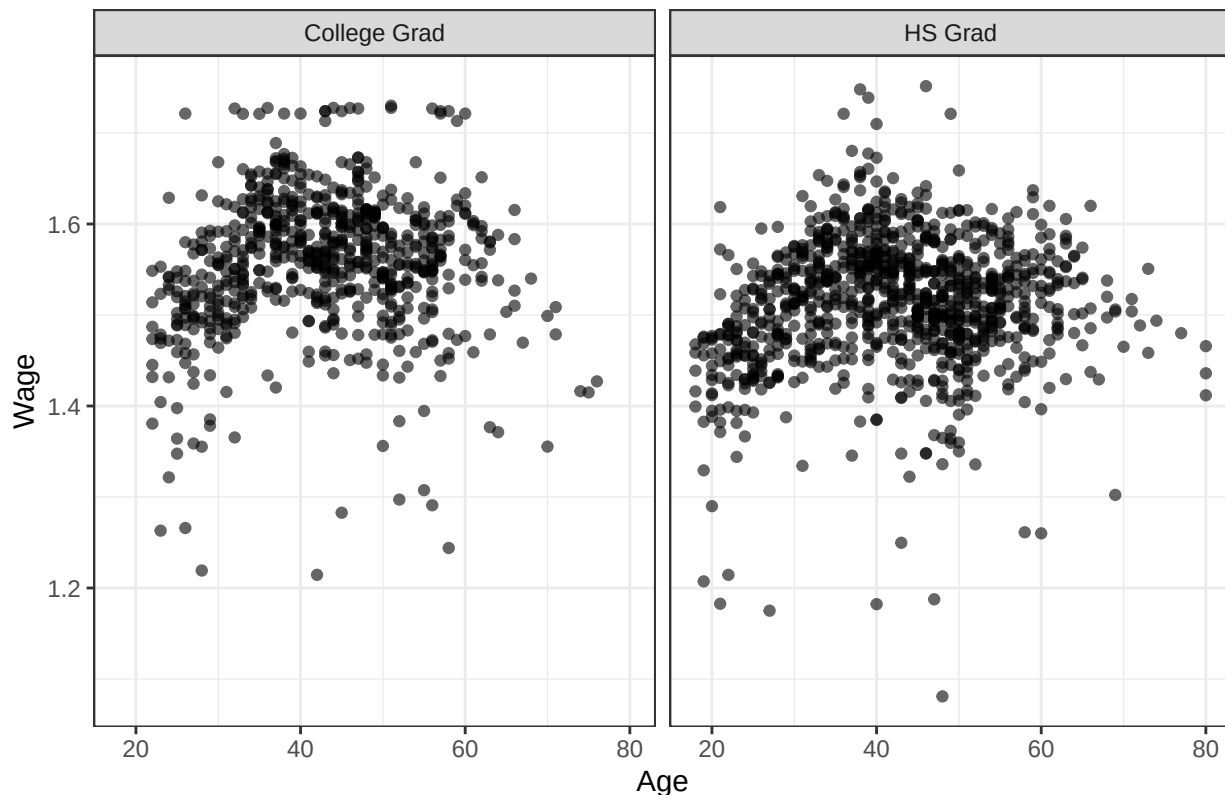
```
##
## College Grad      HS Grad
##           685           971
```

```
# Plot distribution of wage based on education level
ggplot(wage_df, aes(x = education, y=wage)) +
  geom_boxplot() +
  labs(
    title = "Wage distribution by education",
    x = "Education level",
    y = "Wage"
  )
```



```
# plot relationship between age and wage
ggplot(wage_df, aes(x = age, y = wage)) +
  geom_point(alpha = 0.6) +
  facet_wrap(~education) +
  labs(
    title = "Wage vs Age by Education Level",
    x = "Age",
    y = "Wage"
  )
```

Wage vs Age by Education Level



There are 971 HS Grad and 685 College Grad. For wages among different educational level, College Grad have a higher median wage compared with High School Grad; the wage variance among college grad is also smaller than those of high school grad, particularly in the age group between 40 to 60

1.4 Prepare for modeling (1 point)

```
set.seed(4710) # set seed for reproducibility
n <- nrow(wage_df)
train_samples <- sample(1:n, size = 0.8 * n, replace = FALSE)
wage_train = wage_df[train_samples,]
wage_test = wage_df[-train_samples,]
```

1.5 Validation set approach to model selection (33 points)

1.5.1 Data splitting (3 points)

Solution.

```
# Split train_test based on education level
wage_train_HS = wage_train[wage_train$education == "HS Grad",]
wage_train_coll = wage_train[wage_train$education == "College Grad",]
wage_test_HS = wage_test[wage_test$education == "HS Grad",]
wage_test_coll = wage_test[wage_test$education == "College Grad",]
```

```

# Create learning and validation set
set.seed(4710) # set seed for reproducibility
HS_n <- nrow(wage_train_HS)
Coll_n = nrow(wage_train_coll)
HS_learn_samples <- sample(1:HS_n, size = 0.7 * HS_n, replace = FALSE)
coll_learn_samples <- sample(1:Coll_n, size = 0.7 * Coll_n, replace = FALSE)

wage_learn_HS = wage_train_HS[HS_learn_samples,]
wage_val_HS = wage_train_HS[-HS_learn_samples,]
wage_learn_coll = wage_train_coll[coll_learn_samples,]
wage_val_coll = wage_train_coll[-coll_learn_samples,]

```

1.5.2 Estimate MSE given a df (15 points)

Solution.

```

library(splines)
validation_mse <- function(learn_data, val_data, df) {
  spline_fit <- lm(wage ~ ns(age, df=df), data = learn_data)
  predictions <- predict(spline_fit, newdata = val_data)
  mse <- mean((predictions - val_data$wage)^2)
  return(mse)
}
df_seq = seq(2,20,1)
val_errors_HS = numeric(length(df_seq))
val_errors_coll = numeric(length(df_seq))
for (i in 1:length(df_seq)) {
  val_errors_HS[i] = validation_mse(wage_learn_HS, wage_val_HS, df_seq[i])
  val_errors_coll[i] = validation_mse(wage_learn_coll, wage_val_coll, df_seq[i])
}
print(val_errors_HS)

## [1] 0.00431 0.00424 0.00429 0.00421 0.00419 0.00421 0.00420 0.00420 0.00420
## [10] 0.00421 0.00418 0.00421 0.00417 0.00421 0.00418 0.00417 0.00420 0.00422
## [19] 0.00424

```

```
print(val_errors_coll)
```

```

## [1] 0.00591 0.00563 0.00560 0.00565 0.00562 0.00566 0.00560 0.00557 0.00560
## [10] 0.00567 0.00569 0.00575 0.00585 0.00580 0.00579 0.00578 0.00585 0.00581
## [19] 0.00577

```

1.5.3 Model selection based on estimated MSE (10 points)

Solution.

```

# plots for HS grad degree of freedom vs. mse
mse_df <- data.frame(
  df = rep(df_seq, 2),
  mse = c(val_errors_HS, val_errors_coll),

```

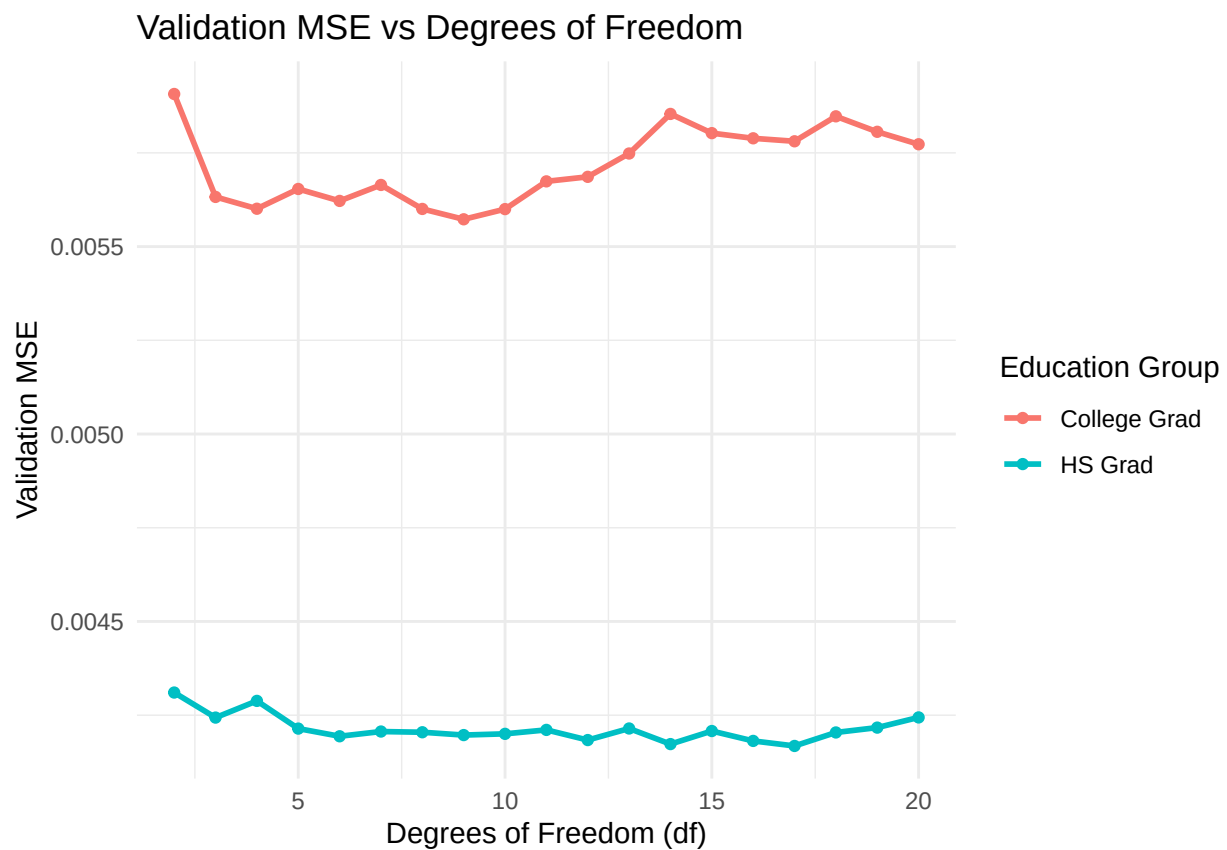


```

group = rep(c("HS Grad", "College Grad"), each = length(df_seq))
)

ggplot(mse_df, aes(x = df, y = mse, color = group)) +
  geom_line(size = 1) +
  geom_point() +
  labs(
    title = "Validation MSE vs Degrees of Freedom",
    x = "Degrees of Freedom (df)",
    y = "Validation MSE",
    color = "Education Group"
  ) +
  theme_minimal()

```



For HS Grad group, $df=14$ minimizes the validation MSE; for College Grad, $df=4$ minimizes the validation MSE.

1.5.4 Fit chosen model and assessment (5 points)

Solution.

```

df_HS = 14
df_coll = 4
HS_spline_fit = lm(wage~ns(age, df = df_HS), data = wage_train_HS)
coll_spline_fit = lm(wage~ns(age, df = df_coll), data = wage_train_coll)

```

```

HS_spline_predict = predict(HS_spline_fit, newdata=wage_test_HS)
coll_spline_predict = predict(coll_spline_fit, newdata=wage_test_coll)
HS_mse = mean((HS_spline_predict - wage_test_HS$wage)^2)
coll_mse = mean((coll_spline_predict - wage_test_coll$wage)^2)
print(HS_mse)

```

```
## [1] 0.00479
```

```
print(coll_mse)
```

```
## [1] 0.00486
```

Spline with best df from validation set approach for HS Grad is 0.00479 and for College Grad is 0.00486

1.6 Cross validation model selection (18 points)

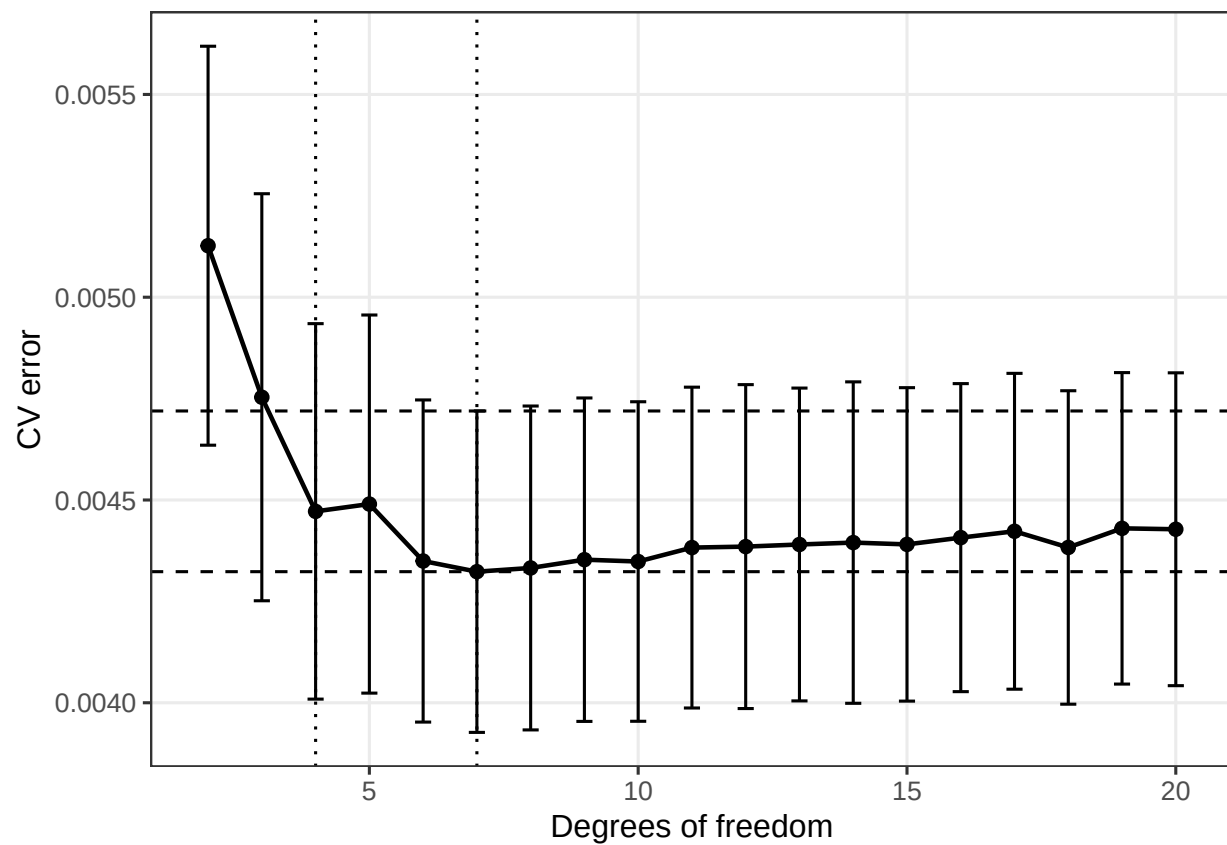
1.6.1 Model selection (11 points)

Solution.

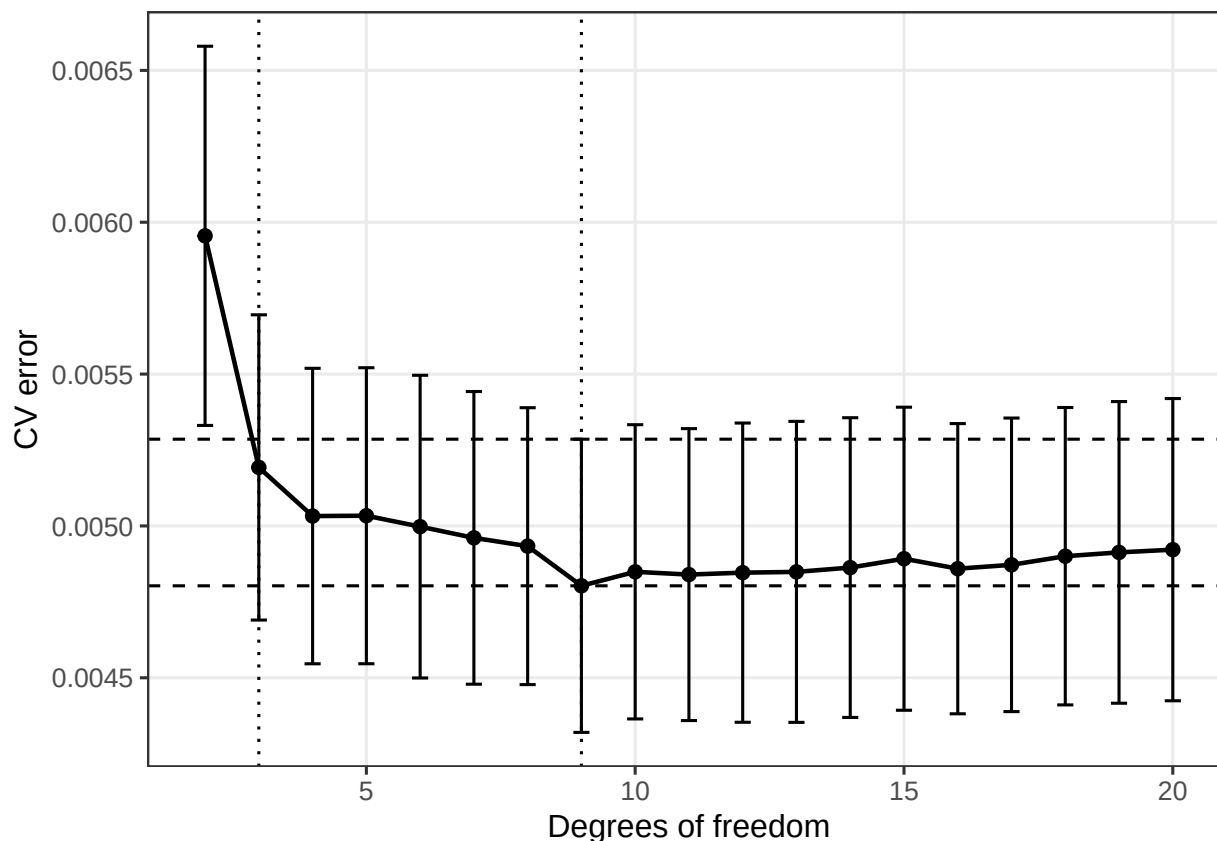
```

cv_HS = cross_validate_spline(wage_train_HS$age,
                              wage_train_HS$wage,
                              nfolds = 10,
                              df_values = 2:20,
                              seed = 4710)
cv_coll = cross_validate_spline(wage_train_coll$age,
                                wage_train_coll$wage,
                                nfolds = 10,
                                df_values = 2:20,
                                seed = 4710)
print(cv_HS$cv_plot)

```



```
print(cv_coll$cv_plot)
```



For HS Grad cv approach, the df.min is 7 and df.1se is 4; for Coll Grad cv approach, df.min is 9 and df.1se is 3. Based on the cv plot, I would choose df=4 for HS Grad spline and df=3 for college Grad spline. The validation set approach has resulting best df different from CV approach. Based on the MSE plots on both approaches, I would choose df=4 for HS Grad and df=3 for Coll Grad, since higher df does not provide significant better MSE on both approaches.

1.6.2 Final fit (5 points)

Solution.

```
df_hs_cv = 4
df_coll_cv = 3
HS_spline_cv = lm(wage~ns(age, df = df_hs_cv), data = wage_train_HS)
coll_spline_cv = lm(wage~ns(age, df = df_coll_cv), data = wage_train_coll)

# get testing mse
HS_spline_predict_cv = predict(HS_spline_cv, newdata=wage_test_HS)
coll_spline_predict_cv = predict(coll_spline_cv, newdata=wage_test_coll)
HS_mse_cv = mean((HS_spline_predict_cv - wage_test_HS$wage)^2)
coll_mse_cv = mean((coll_spline_predict_cv - wage_test_coll$wage)^2)
print(HS_mse_cv)
```

```
## [1] 0.00452
```

```
print(coll_mse_cv)
```

```
## [1] 0.00488
```

```
# also extract training mse
HS_train_preds <- predict(HS_spline_cv, newdata = wage_train_HS)
HS_train_mse <- mean((wage_train_HS$wage - HS_train_preds)^2)
coll_train_preds <- predict(coll_spline_cv, newdata = wage_train_coll)
coll_train_mse <- mean((wage_train_coll$wage - coll_train_preds)^2)
print(HS_train_mse)
```

```
## [1] 0.00444
```

```
print(coll_train_mse)
```

```
## [1] 0.00495
```

I chose `df.1se` for both education levels. Training and testing MSE for `HS_Grad` is 0.00444 and 0.00452; training and testing MSE for `Coll_Grad` is 0.00495 and 0.00488. The training error is smaller than testing error for both HS Grad and Coll Grad; however, since training and testing error do not differ much, the occurred overfitting is not that substantial.

1.6.3 Compare with validation set approach (2 points)

Solution. Validation set approach has test MSE 0.00479 for HS Grad and 0.00486 for College Grad. CV approach has test MSE 0.00452 for HS Grad and 0.00488 for College Grad. In HS Grad case, degree of freedom (`df`) chosen by CV is better than validation set approach; In College Grad case, `df` chosen by validation set is slightly better than CV.

2 Case study: kNN classification with Default data (36 points)

```
library(ISLR2)
data("Default")
```

2.1 Data split (2 points)

Solution.

```
set.seed(4710) # set seed for reproducibility
n_default <- nrow(Default)
train_samples_d <- sample(1:n_default, size = 0.8 * n_default, replace = FALSE)
default_train = Default[train_samples_d,]
default_test = Default[-train_samples_d,]
```

2.2 Class imbalance and baseline (6 points)

Solution.

```
# check distribution of training dataset
print(table(default_train$default))
```

```
##
##   No   Yes
## 7740  260
```

```
# check majority class
majority_class = names(which.max(table(default_train$default)))
print(majority_class)
```

```
## [1] "No"
```

```
# make baseline predictions
baseline_preds = rep(majority_class, nrow(default_test))

# check accuracy
baseline_accuracy = mean(baseline_preds == default_test$default)
print(baseline_accuracy)
```

```
## [1] 0.964
```

The “No” class has 7740 data points, which is far more than “Yes” class in the training dataset. For the baseline classifier which always report “No”, the accuracy on test dataset is 0.964. However, since in this case false negative have a huge impact on the prediction results, accuracy is not a good metric and a classifier only predict no is not a good one.

2.3 kNN classification (28 points)

2.3.1 Load results

Solution.

```
knn_counts = read_csv("knn_counts_by_K.csv")

## Rows: 60 Columns: 7
## -- Column specification -----
## Delimiter: ","
## chr (1): split
## dbl (6): K, TP, FP, TN, FN, seed
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

2.3.2 Compute error metrics in classification (4 points)

Solution.

```
# calculate accuracy, precision, recall and F1_score
knn_counts$accuracy = (knn_counts$TN + knn_counts$TP) / (knn_counts$TN + knn_counts$TP + knn_counts$FN)
knn_counts$precision = knn_counts$TP / (knn_counts$TP + knn_counts$FP)
knn_counts$recall = knn_counts$TP / (knn_counts$TP + knn_counts$FN)
knn_counts$F1_score = 2 * (knn_counts$precision * knn_counts$recall) / (knn_counts$precision + knn_counts$recall)
print(knn_counts)
```

```
## # A tibble: 60 x 11
##   split      K    TP    FP    TN    FN  seed accuracy precision recall F1_score
##   <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>   <dbl>   <dbl> <dbl>   <dbl>
## 1 learn~    1   40    0 1080    0    1     1     1     1     1
## 2 test     1    3   15   374    8    1  0.942  0.167  0.273  0.207
## 3 valid~   1    6    9   452   13    1  0.954  0.4    0.316  0.353
## 4 learn~   2   30   12  1068   10    1  0.980  0.714  0.75   0.732
## 5 test     2    3   13   376    8    1  0.948  0.188  0.273  0.222
## 6 valid~   2    7    8   453   12    1  0.958  0.467  0.368  0.412
## 7 learn~   3   23    9  1071   17    1  0.977  0.719  0.575  0.639
## 8 test     3    3    4   385    8    1  0.97   0.429  0.273  0.333
## 9 valid~   3    5    5   456   14    1  0.960  0.5    0.263  0.345
## 10 learn~  4   20   12  1068   20    1  0.971  0.625  0.5    0.556
## # i 50 more rows
```

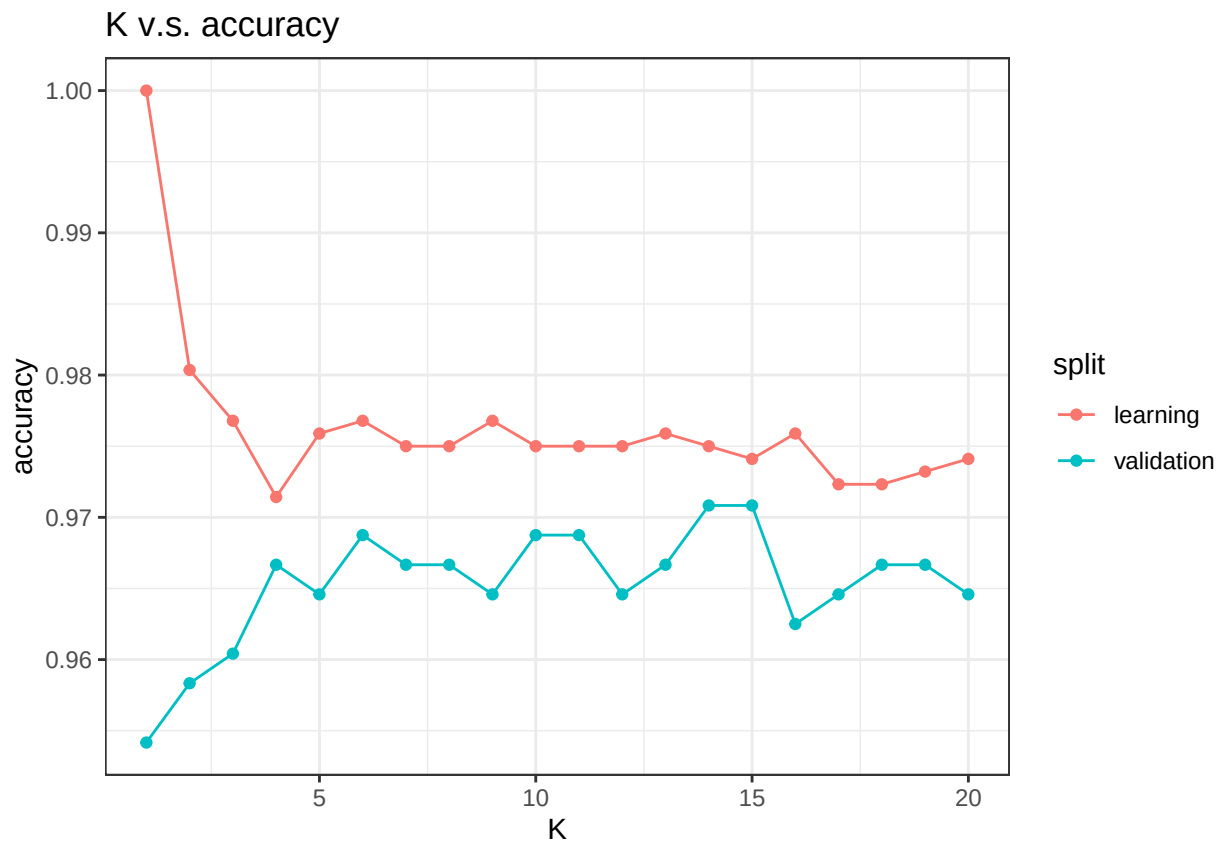
2.3.3 Plot metrics vs. K (6 points)

Solution.

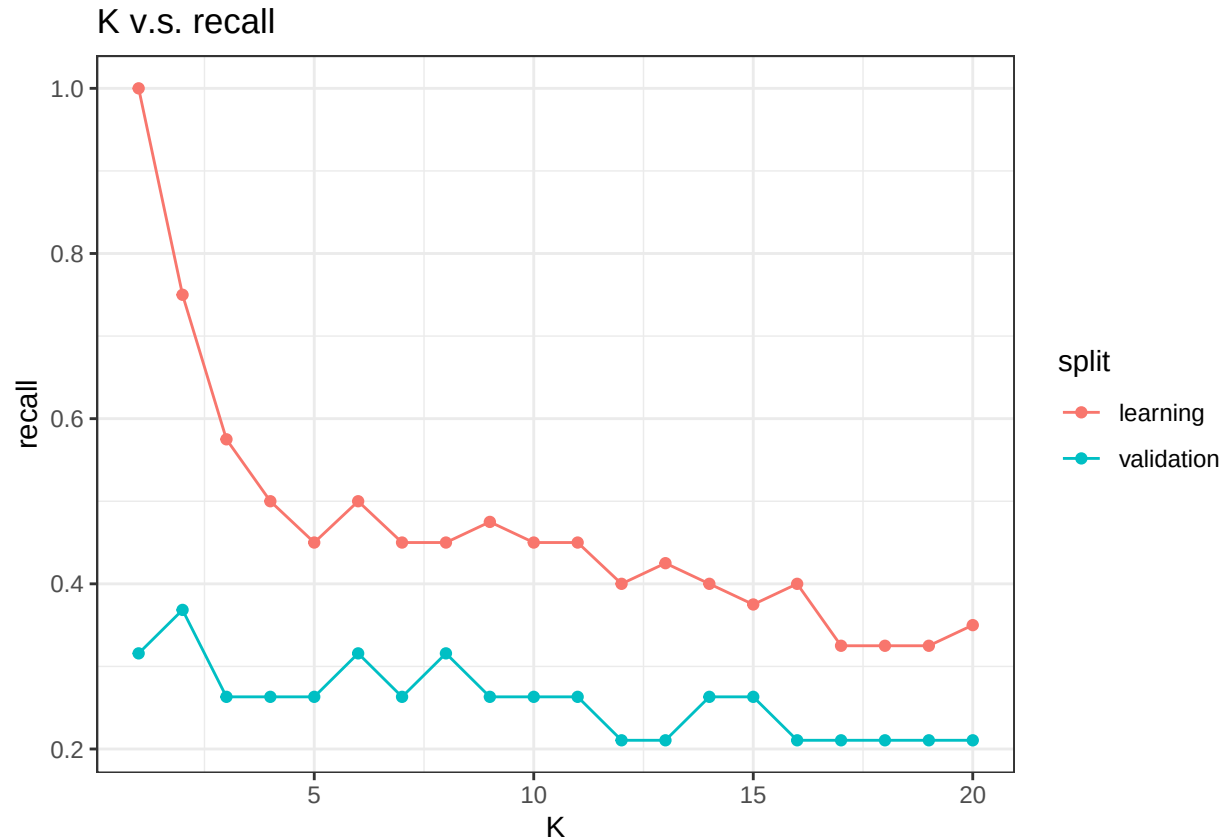
```
# get only learning and validation split
knn_training = knn_counts[knn_counts$split %in% c("learning", "validation"),]

# plot based on accuracy
```

```
ggplot(data=knn_training, aes(x = K, y=accuracy, color = split)) +
  geom_line() + geom_point() +
  labs(
    title = "K v.s. accuracy",
    x = "K",
    y = "accuracy",
    color = "split"
  )
)
```



```
# plot based on recall
ggplot(data=knn_training, aes(x = K, y=recall, color = split)) +
  geom_line() + geom_point() +
  labs(
    title = "K v.s. recall",
    x = "K",
    y = "recall",
    color = "split"
  )
)
```

I noticed the recall is much less than the accuracy level for both learning and validation data. The accuracy level is higher than 0.95 but recall level are below 0.4 for validation dataset. Based on recall formula $TP / (TP + FN)$, one problem of the KNN approach is the large number of FN with respect to TP, resulting a low recall rate.

2.3.4 Model selection and improvement (conceptual) (8 points)

Solution. As a risk manager, I would prefer recall as a metric than accuracy, since I am more concerned with the proportion of false negative that is of a high cost. Using recall and select a K based on validation approach is not a good idea, since we should also consider precision, which can inform when we have much false positives; otherwise, we may come up with a model classifies everything to “Yes”. Also, the majority vote for KNN might be an issue since in default data, most of data are “No”, only handful are “Yes” and the majority vote mechanism will make KNN more likely to classify “No” as an answer for default. To solve this issue, we can consider adding weights to “No” answer during training to combat the class imbalance problem.

2.3.5 Model complexity (conceptual) (10 points)

Solution. The model complexity increases as K decreases, since a smaller K will make the model more sensitive to slightly change in neighborhood area, hence increase model complexity. The $df \propto n/K$ for KNN model makes sense because the model complexity is inverse proportional with K and proportional with n as more data in the training dataset will make the KNN model to get more diversified predictions based on training data points, which increases model complexity. When K is increased, the model is less likely be influenced by noise in training data, which

decreases the variance of the model hence getting better predictions. However, increasing K too large may make the model oversimplified, which increase the bias of the model, hence leading to worse predictions.